

ไฟล์ต่าง ๆ ในระบบ ทำงานอย่างไร

Interview Manager — สถาปัตยกรรมและหน้าที่ของแต่ละไฟล์

อธิบายโครงสร้างโปรเจกต์ ตั้งแต่ไฟล์ตั้งค่า ฝั่งเซิร์ฟเวอร์ (Express + SQLite) ฝั่งหน้าเว็บ (React) ไปจนถึงลำดับการทำงานเมื่อผู้ใช้กดปุ่ม
หนึ่งครั้ง พร้อมข้อสังเกตที่ควรรู้ก่อนแก้ไขโค้ด

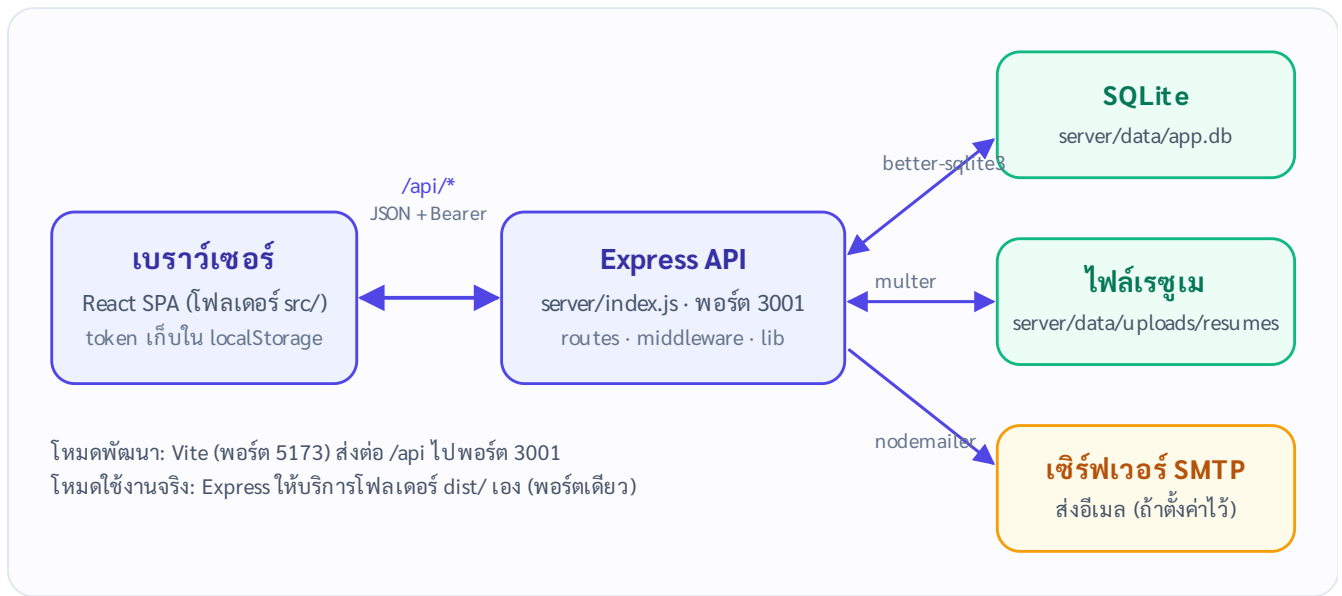
สารบัญ

บทที่ 1 ภาพรวมสถาปัตยกรรม	3
1.1 ระบบนี้ประกอบด้วยอะไรบ้าง	3
1.2 เทคโนโลยีที่ใช้	3
1.3 แผนผังโฟลเดอร์	4
1.4 ตัวอย่าง: กดปุ่ม “Schedule interview” เกิดอะไรขึ้น	4
บทที่ 2 ไฟล์ระดับรากและไฟล์ตั้งค่า	5
2.1 ไฟล์ตั้งค่าโปรเจกต์	5
2.2 คำสั่ง npm ใน package.json	6
2.3 ไฟล์สำหรับ Docker และ CI	6
บทที่ 3 ฟังก์ชันเซิร์ฟเวอร์ (server/)	8
3.1 server/index.js — จุดเริ่มต้น	8
3.2 ฐานข้อมูล: db.js, schema.sql, seed.js	8
3.3 middleware/auth.js — ตรวจสอบสิทธิ์	10
3.4 routes/ — เส้นทาง API	10
3.5 lib/ — ฟังก์ชันช่วยฟังก์ชันเซิร์ฟเวอร์	12
3.6 ชุดทดสอบ (*.test.js และ test/helpers.js)	12
บทที่ 4 ฟังก์ชันเว็บ (src/)	14
4.1 ลำดับการเริ่มทำงานของหน้าเว็บ	14
4.2 การกำหนดเส้นทางและการป้องกันหน้า	14
4.3 context/ — ข้อมูลกลางและสถานะล็อกอิน	15
4.4 lib/ และ types/	16
4.5 pages/ — หน้าจอ	17
4.6 components/ — ชิ้นส่วนที่ใช้ซ้ำ	18
บทที่ 5 กระบวนการสำคัญที่ไฟล์ทำงานร่วมกัน	20
5.1 การเข้าสู่ระบบและการกู้ session	20
5.2 การบันทึกผลสัมภาษณ์	20
5.3 เรซูเม่: อัปโหลดและเปิดดู	20
5.4 อีเมลแจ้งเตือน	21
บทที่ 6 ข้อสังเกตและข้อควรระวังจากการอ่านโค้ด	22
บทที่ 7 ต้องการแก้เรื่องนี้ ควรเปิดไฟล์ไหน	23

1 ภาพรวมสถาปัตยกรรม

1.1 ระบบนี้ประกอบด้วยอะไรบ้าง

Interview Manager แบ่งเป็น **สองส่วนที่คุยกันผ่าน HTTP** ส่วนแรกคือหน้าเว็บ (Single-Page Application) ที่เขียนด้วย React อยู่ในโฟลเดอร์ `src/` ส่วนที่สองคือเซิร์ฟเวอร์ API เขียนด้วย Express อยู่ในโฟลเดอร์ `server/` ข้อมูลทั้งหมดเก็บในฐานข้อมูล SQLite ไฟล์เดียว และไฟล์เรซูเม่ที่อัปโหลดเก็บเป็นไฟล์บนดิสก์



1.2 เทคโนโลยีที่ใช้

ส่วน	เครื่องมือ	ใช้ทำอะไร
หน้าเว็บ	React 19, TypeScript, Vite	สร้างหน้าจอ ตรวจสอบชนิดข้อมูล และ build เป็นไฟล์สำหรับเผยแพร่
การจัดหน้า	Tailwind CSS v4, clsx, lucide-react	สไตล์แบบ utility class (ตั้งค่าใน CSS ไม่มี tailwind.config) รวม class อย่างมีเงื่อนไข และชุดไอคอน
การนำทาง / กราฟ	react-router 8, recharts	กำหนด URL ของแต่ละหน้า และวาดกราฟเส้นบน Dashboard
เซิร์ฟเวอร์	Express 5, better-sqlite3	รับคำขอ /api และอ่าน/เขียนฐานข้อมูลแบบ synchronous
ความปลอดภัย	bcryptjs, helmet, cors, express-rate-limit	เข้ารหัสผ่าน ตั้ง HTTP header ป้องกันจำกัดโดเมน และจำกัดจำนวนครั้งล็อกอิน
ไฟล์และอีเมล	multer, nodemailer, dotenv	รับไฟล์อัปโหลด ส่งอีเมล และโหลดค่าจากไฟล์ .env

ส่วน	เครื่องมือ	ใช้ทำอะไร
คุณภาพโค้ด	Vitest, supertest, oxlint	ทดสอบ API ฝั่งเซิร์ฟเวอร์ และตรวจโค้ดฝั่งหน้าเว็บ

1.3 แผนผังโฟลเดอร์

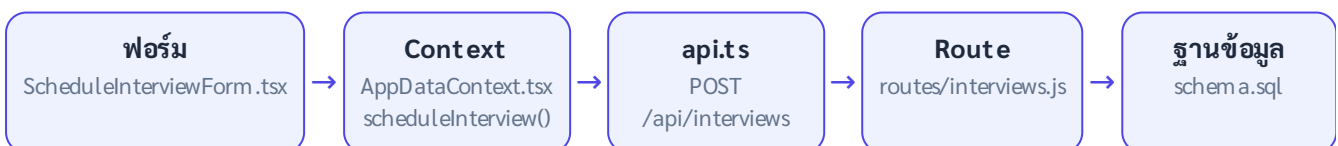
```

Int_claude/
├─ index.html          หน้า HTML เปล่าที่โหลด src/main.tsx
├─ package.json        รายการแพ็คเกจและคำสั่ง npm
├─ vite.config.ts · vitest.config.ts · tsconfig*.json · .oxlintrc.json
├─ Dockerfile · .dockerignore · .gitignore · .env.example
├─ .github/workflows/ci.yml      ตรวจอัตโนมัติบน GitHub
├─ public/favicon.svg   ไอคอนของเว็บ
├─ src/                  * ฝั่งหน้าเว็บ (React)
│   ├── main.tsx · App.tsx · routes.tsx · index.css
│   ├── pages/           หน้าจอแต่ละหน้า (1 ไฟล์ = 1 route)
│   ├── components/      ชิ้นส่วน UI (auth, layout, candidates, interviews,
│   │                   interviewers, dashboard, ui)
│   ├── context/         ที่เก็บข้อมูลกลาง + สถานะการเข้าสู่ระบบ
│   ├── lib/             ฟังก์ชันช่วย (api, date, status, roleHome, thresholds)
│   └── types/           ชนิดข้อมูล TypeScript
├─ server/               * ฝั่งเซิร์ฟเวอร์ (Express)
│   ├── index.js         จุดเริ่มต้นของเซิร์ฟเวอร์
│   ├── db.js · schema.sql · seed.js
│   ├── routes/          auth, candidates, interviews, users,
│   │                   evaluation-template, activity-log (+ ไฟล์ .test.js)
│   ├── middleware/auth.js
│   ├── lib/             activityLog, evaluation, ids, mailer, serialize, uploads
│   ├── test/helpers.js  ตัวช่วยสำหรับเขียนเทสต์
│   └── data/            app.db และ uploads/ (สร้างตอนรัน ไม่อยู่ใน git)
└─ dist/                 ผลลัพธ์จาก npm run build

```

1.4 ตัวอย่าง: กดปุ่ม “Schedule interview” เกิดอะไรขึ้น

ตัวอย่างนี้แสดงให้เห็นว่าไฟล์ต่าง ๆ ทำงานต่อกันเป็นลำดับอย่างไร เพราะเกือบทุกพีเจอรืในระบบใช้รูปแบบเดียวกัน



- 1 ฟอร์ม** รวมค่าที่กรอก (ผู้สมัคร ผู้สัมภาษณ์ วันเวลา ฯลฯ) แปลงวันเวลาเป็นรูปแบบ ISO แล้วเรียก `scheduleInterview()` จาก `useAppData()`
- 2 AppDataContext.tsx** เรียก `api.post('/interviews', input)` — `api.ts` ใส่ `Authorization: Bearer <token>` จาก `localStorage` และต่อคำนำหน้า `/api` ให้
- 3 server/index.js** ส่งคำขอเข้า `routes/interviews.js` ซึ่งผ่าน `requireAuth` (ตรวจ token ในตาราง `sessions`) และ `requireRole('admin')` ก่อน

- 4 handler ตรวจสอบข้อมูลที่จำเป็น แล้วใน **transaction เดียว** เพิ่มแถวใน `interviews` และ `interview_interviewers` พร้อมเปลี่ยนสถานะผู้สมัครเป็น Interview Scheduled (ยกเว้น Offer/Rejected)
- 5 เรียก `logActivity()` บันทึกลง `activity_log` และส่งอีเมลยืนยันให้ผู้สมัครแบบ fire-and-forget (ไม่รอผล) ผ่าน `lib/mailer.js`
- 6 ตอบกลับ `{ interview, candidate }` ที่ผ่าน `serialize.js` แล้ว Context ส่ง `dispatch` สองครั้ง (`ADD_INTERVIEW` และ `UPDATE_CANDIDATE`) หน้าจอทุกหน้าที่ใช้ข้อมูลนี้จะอัปเดตพร้อมกัน

2 ไฟล์ระดับรากและไฟล์ตั้งค่า

ไฟล์เหล่านี้อยู่นอกโฟลเดอร์ `src/` และ `server/` ทำหน้าที่กำหนดวิธี build ทดสอบ ตรวจสอบโค้ด และนำขึ้นใช้งาน

2.1 ไฟล์ตั้งค่าโปรเจกต์

ไฟล์	หน้าที่และวิธีทำงาน
<code>package.json</code>	ประกาศชื่อโปรเจกต์ (<code>int-claude</code>) แพคเกจที่ใช้ และคำสั่ง <code>npm</code> ทั้งหมด (ดูหัวข้อ 2.2) มี <code>"type": "module"</code> ทำให้ไฟล์ <code>.js</code> ทั้งฝั่งเซิร์ฟเวอร์ใช้ <code>import/export</code> แบบ ES Module
<code>package-lock.json</code>	ล็อกเวอร์ชันแพคเกจทุกตัวให้ติดตั้งได้ผลเหมือนกันทุกเครื่อง (<code>npm ci</code> ใช้ไฟล์นี้) ไม่ต้องแก้ด้วยมือ
<code>index.html</code>	หน้า HTML เดียวของแอป มีเพียง <code><div id="root"></code> กับสคริปต์ <code>/src/main.tsx</code> ที่ React จะมาสร้างหน้าจอทั้งหมดลงใน div นี้ (SPA)
<code>vite.config.ts</code>	เปิดปลั๊กอิน React และ Tailwind และตั้ง proxy ให้คำขอ <code>/api</code> ตอนพัฒนาถูกส่งต่อไป <code>http://localhost:3001</code> ทำให้เบราว์เซอร์มองว่าเป็นโดเมนเดียวกัน (ไม่ต้องตั้ง CORS)
<code>vitest.config.ts</code>	บอก Vitest ให้หาเทสต์ที่ <code>server/**/*.test.js</code> และตั้ง <code>DB_PATH=:memory:</code> เพื่อให้เทสต์ใช้ฐานข้อมูลชั่วคราวในหน่วยความจำ ไม่แตะข้อมูลจริง
<code>tsconfig.json</code> , <code>tsconfig.app.json</code> , <code>tsconfig.node.json</code>	ตัวแรกเป็นตัวชี้ไปยังอีกสองตัว: <code>app</code> ใช้ตรวจโค้ดใน <code>src/</code> (โหมดเข้มงวด เช่น ห้ามตัวแปรที่ไม่ได้ใช้ และต้องใช้ <code>import type</code> สำหรับชนิดข้อมูล) ส่วน <code>node</code> ใช้ตรวจ <code>vite.config.ts</code>
<code>.oxlintrc.json</code>	ตั้งค่า <code>oxlint</code> (ปลั๊กอิน <code>react</code> , <code>typescript</code> , <code>oxc</code>) เน้นกฎ <code>rules-of-hooks</code> เป็น error และเตือนเรื่องการ <code>export</code> คอมโพเนนต์ปนกับค่าอื่น — ตรวจสอบเฉพาะฝั่งหน้าเว็บ ไม่ตรวจ <code>server/</code>

ไฟล์	หน้าที่และวิธีทำงาน
<code>.env.example / .env</code>	<code>.env.example</code> เป็นแม่แบบ (SMTP_*, APP_TIMEZONE, CORS_ORIGIN) ส่วน <code>.env</code> คือไฟล์ที่ใช้จริงซึ่งเก็บค่าลับ <code>server/index.js</code> โหลดด้วย <code>dotenv</code> ตอนเริ่มทำงาน (ไม่ได้เปิดอ่านเนื้อหา <code>.env</code> ในการจัดทำเอกสารนี้)
<code>README.md</code>	ข้อความมาตรฐานของแม่แบบ Vite + React (ยังไม่ได้เขียนเกี่ยวกับระบบนี้)
<code>CLAUDE.md</code>	บันทึกแนวทางของโปรเจกต์สำหรับผู้ช่วยเขียนโค้ด: คำสั่ง สถาปัตยกรรม ข้อตกลงการเขียน เป็นประโยชน์กับนักพัฒนาใหม่เช่นกัน
<code>public/favicon.svg</code>	ไอคอนของแท็บเบราว์เซอร์ Vite คัดลอกเข้า <code>dist/</code> ตอน build
<code>.gitignore</code>	ไม่ให้ git เก็บ <code>node_modules</code> , <code>dist</code> , <code>.env</code> , ฐานข้อมูล <code>server/data/*.db</code> และไฟล์เรซูเม่ที่อัปโหลด
<code>dist/ , node_modules/</code>	สร้างโดยเครื่องมือ: <code>dist</code> คือผลลัพธ์ของ <code>npm run build</code> (ไฟล์ JS/CSS ที่บีบอัดแล้ว) ส่วน <code>node_modules</code> คือแพ็คเกจที่ติดตั้งไม่ต้องแก้ไข

2.2 คำสั่ง npm ใน package.json

คำสั่ง	ทำอะไร
<code>npm run dev</code>	เปิด Vite dev server (หน้าเว็บอย่างเดียว) พร้อม hot reload
<code>npm run server</code>	เปิด API ด้วย <code>node server/index.js</code> (พอร์ต 3001)
<code>npm run dev:all</code>	เปิดทั้งสองอย่างพร้อมกันด้วย <code>concurrently</code>
<code>npm run db:seed</code>	รัน <code>server/seed.js</code> ใส่ข้อมูลตัวอย่าง (ทำงานเฉพาะเมื่อยังไม่มีผู้ใช้)
<code>npm run build</code>	ตรวจชนิดข้อมูลด้วย <code>tsc -b</code> แล้ว <code>vite build</code> ได้ไฟล์เดอร์ <code>dist/</code>
<code>npm run lint</code>	ตรวจโค้ดด้วย <code>oxlint</code>
<code>npm test / test:watch</code>	รันชุดทดสอบฝั่งเซิร์ฟเวอร์ด้วย Vitest (รอบเดียว / ต่อเนื่อง)
<code>npm run preview</code>	ดูตัวอย่างหน้าเว็บที่ build แล้ว

2.3 ไฟล์สำหรับ Docker และ CI

ไฟล์	หน้าที่และวิธีทำงาน
<code>Dockerfile</code>	สร้าง image สองขั้น ขั้น build ติดตั้งแพ็คเกจครบแล้วรัน <code>npm run build</code> ขั้น runtime (Node 24) ติดตั้งเฉพาะแพ็คเกจสำหรับใช้งานจริง คัดลอก <code>server/</code> และ <code>dist/</code> ตั้ง <code>NODE_ENV=production</code> ประกาศ <code>VOLUME /app/server/data</code> เปิดพอร์ต 3001 และมี HEALTHCHECK เรียก <code>/healthz</code>

ไฟล์	หน้าที่และวิธีทำงาน
<code>.dockerignore</code>	กันไม่ให้คัดลอก <code>node_modules</code> , <code>dist</code> , <code>server/data</code> ,ไฟล์ <code>.env</code> , <code>.git</code> ฯลฯ เข้าไปใน image
<code>.github/workflows/ci.yml</code>	เมื่อ push เข้า main หรือเปิด pull request GitHub จะรัน <code>npm ci</code> → <code>lint</code> → <code>build</code> → <code>test</code> และอีกงานหนึ่งลอง <code>docker build</code> (สร้าง image เท่านั้น ไม่ได้ลองรัน)

3 ผังเซิร์ฟเวอร์ (server/)

เซิร์ฟเวอร์เขียนด้วย JavaScript ธรรมดา (ES Module) ไม่มีขั้นตอน build ไม่ใช้ ORM ใช้ SQL ตรง ๆ ผ่าน `better-sqlite3` ซึ่งทำงานแบบ synchronous ดังนั้น handler ส่วนใหญ่จึงไม่ต้องใช้ `async/await` (ยกเว้น ส่วนที่ส่งอีเมล)

3.1 server/index.js — จุดเริ่มต้น

ไฟล์นี้สร้างแอป Express และ export ชื่อ `app` (ทดสอบนำไปใช้กับ `supertest` โดยไม่ต้องเปิดพอร์ตจริง) ลำดับการทำงานเป็นดังนี้

- 1 โหลด `.env` ด้วย `dotenv/config` แล้วใส่ `helmet()` เพื่อตั้ง HTTP security header
- 2 ตั้ง `cors` — ค่าเริ่มต้นไม่อนุญาตข้ามโดเมน จะอนุญาตเฉพาะโดเมนที่ระบุใน `CORS_ORIGIN` (คั่นด้วยจุลภาค)
- 3 เปิด `express.json()` อ่านเนื้อหาคำขอแบบ JSON
- 4 เพิ่ม GET `/healthz` (ไม่ต้องล็อกอิน) รัน `SELECT 1` เพื่อดูว่าฐานข้อมูลตอบสนอง ใช้กับ Docker HEALTHCHECK และตัวเฝ้าระบบ
- 5 ผูก router ทั้งหมดที่ `/api/auth`, `/api/candidates`, `/api/interviews`, `/api/users`, `/api/evaluation-template`, `/api/activity-log`
- 6 ถ้า `NODE_ENV=production` และมีโฟลเดอร์ `dist/` จะให้บริการไฟล์หน้าเว็บ และตอบ `index.html` สำหรับทุกเส้นทางที่ไม่ใช่ `/api` (ให้ `react-router` จัดการต่อ)
- 7 ท้ายสุดมี handler ตอบ 404 ({ error: 'Not found.' }) และตัวจับข้อผิดพลาดที่ตอบ 500 โดยไม่เปิดเผยรายละเอียด
- 8 เมื่อไม่ใช่โหมดทดสอบจึงเรียก `app.listen()` (พอร์ตจาก `PORT` หรือ 3001) และปิดฐานข้อมูลอย่างเรียบร้อยเมื่อได้รับสัญญาณ `SIGTERM/SIGINT`

3.2 ฐานข้อมูล: db.js, schema.sql, seed.js

server/db.js

เปิดการเชื่อมต่อ SQLite ที่ `server/data/app.db` (หรือ path จากตัวแปร `DB_PATH` — เทสที่ใช้ `:memory:`) เปิด `foreign_keys` แล้วทำงานตามลำดับทุกครั้งที่เซิร์ฟเวอร์เริ่ม:

- สร้างตาราง โดยอ่านและรัน `schema.sql` (ใช้ `CREATE TABLE IF NOT EXISTS` จึงรันซ้ำได้ปลอดภัย)
- ย้ายโครงสร้างแบบเบา ๆ ด้วย `ensureColumn()` — เพิ่มคอลัมน์ที่ฐานข้อมูลรุ่นเก่ายังไม่มี (`users.position`, `eval_interviewer_signature/position`, `candidates.resume_filename`)
- ใส่แม่แบบเกณฑ์ประเมินเริ่มต้น (3 หัวข้อ 20 เกณฑ์) หากตาราง `evaluation_sections` ยังว่าง
- ย้ายข้อมูลเก่า — ถ้าพบฐานข้อมูลที่เคยเก็บคะแนนแบบ 10 เกณฑ์ตายตัวเป็นคอลัมน์ `eval_*` จะคัดลอกเข้าตาราง `evaluation_scores` ได้หัวข้อ “Legacy Evaluation” (ติดตั้งใหม่จะไม่เข้าเงื่อนไข)

server/schema.sql — ตารางทั้งหมด

ตาราง	ข้อมูลสำคัญ	ความสัมพันธ์
users	id, name, email (ไม่ซ้ำ), password_hash, role (admin/interviewer), position	ผู้ใช้ทั้งแอดมินและผู้สัมภาษณ์อยู่ตารางเดียว
candidates	ชื่อ อีเมล โทรศัพท์ ตำแหน่ง status resume_url notes created_at (+ resume_filename)	status ถูกจำกัดด้วย CHECK 5 ค่า
interviews	candidate_id, date, duration_minutes, type, status, location, notes และ eval_* (total, result, notes, signature, position)	candidate_id → candidates (ON DELETE CASCADE) ผลประเมิน 1 ชุดต่อการสัมภาษณ์ เก็บเป็นคอลัมน์ตรงนี้
interview_interviewers	interview_id, user_id	ตารางเชื่อมหลายต่อหลายระหว่าง interviews กับ users (CASCADE ทั้งสองฝั่ง)
evaluation_sections	id, name, sort_order	หัวข้อของแบบประเมิน
evaluation_criteria	section_id, name, sort_order	section_id → evaluation_sections (CASCADE)
evaluation_scores	interview_id, sort_order, section_name, criterion_name, score	เก็บชื่อหัวข้อ/เกณฑ์ ณ ตอนบันทึก (snapshot) ไม่ผูกกับตารางเกณฑ์ จึงแก้ไขแบบภายหลังได้โดยรายงานเก่าไม่เปลี่ยน
sessions	token (คีย์หลัก), user_id, expires_at	โทเค็นการเข้าสู่ระบบ (CASCADE เมื่อลบผู้ใช้)
activity_log	actor_id, actor_name, action, entity_type/id/label, details, created_at	เก็บชื่อผู้กระทำและชื่อรายการเป็น snapshot ประวัติจึงอ่านได้แม้ผู้ใช้หรือรายการถูกลบไปแล้ว

server/seed.js

ใช้ผ่าน `npm run db:seed` — นำเข้า `db.js` (ได้แม่แบบเกณฑ์อัตโนมัติ) ถ้ามีผู้ใช้อยู่แล้วจะจบทันที มิฉะนั้นใน transaction เดียวจะเพิ่มบัญชีตัวอย่าง (แอดมิน 1 ผู้สัมภาษณ์ 4) ผู้สมัคร 12 คน การสัมภาษณ์พร้อมคะแนน ตัวอย่างที่ผูกกับเกณฑ์ตามลำดับ และรหัสผ่านถูกเข้ารหัสด้วย bcrypt ก่อนบันทึก

3.3 middleware/auth.js — ตรวจสอบสิทธิ์

ฟังก์ชัน	ทำอะไร
<code>requireAuth</code>	อ่าน header <code>Authorization: Bearer <token></code> ค้นตาราง <code>sessions</code> (JOIN กับ <code>users</code>) ถ้าไม่พบหรือหมดอายุตอบ 401 ถ้าผ่านจะใส่ <code>req.user = { id, name, email, role }</code> ให้ handler ถัดไป โทเค็นเป็นสตริงสุ่มที่บ่งชี้ผู้ใช้ ไม่ใช่ JWT จึงเพิกถอนได้ทันทีโดยลบแถว
<code>requireRole('admin')</code>	ตรวจสอบ <code>req.user.role</code> ถ้าไม่ตรงตอบ 403 “You do not have permission...” ใช้ต่อท้าย <code>requireAuth</code>

3.4 routes/ — เส้นทาง API

หนึ่งไฟล์ต่อหนึ่งทรัพยากร ทุกไฟล์ยกเว้น `/api/auth/login` เรียก `requireAuth` ก่อน คอลัมน์ “สิทธิ์” ระบุว่าใครเรียกได้

เส้นทาง	สิทธิ์	การทำงาน
routes/auth.js → /api/auth		
<code>POST /login</code>	ทุกคน	จำกัด 10 ครั้ง/15 นาที ตรวจสอบอีเมลและรหัสผ่านด้วย <code>bcrypt</code> สร้างโทเค็น <code>session-<uuid></code> เก็บใน <code>sessions</code> และตอบ <code>{ token, user }</code>
<code>POST /logout</code>	ล็อกอินแล้ว	ลบโทเค็นปัจจุบันออกจากตาราง <code>sessions</code>
<code>GET /me</code>	ล็อกอินแล้ว	ตอบข้อมูลผู้ใช้ของโทเค็น (ใช้ถ้าสถานะตอนเปิดหน้าเว็บใหม่)
<code>DELETE /sessions</code>	ล็อกอินแล้ว	“ออกจากระบบทุกอุปกรณ์” ลบทุกโทเค็นของผู้ใช้ และบันทึกกิจกรรม
routes/candidates.js → /api/candidates		
<code>GET /</code>	ล็อกอินแล้ว	แอดมินเห็นทั้งหมด (รองรับ <code>search, status, limit, offset</code> และ header <code>X-Total-Count</code>) ผู้สัมภาษณ์เห็นเฉพาะผู้สมัครที่ตนถูกมอบหมาย
<code>POST /, PATCH /:id, DELETE /:id</code>	admin	เพิ่ม/แก้ไข/ลบผู้สมัคร ตรวจสอบอีเมลซ้ำ (409) บันทึกกิจกรรม (แยก <code>status_changed</code> ออกจาก <code>updated</code>) การลบฟ่งลบการสัมภาษณ์ด้วย <code>CASCADE</code> และลบไฟล์เรซูเม่ออกจากดิสก์
<code>POST /:id/resume</code>	admin	รับไฟล์ผ่าน <code>multer</code> (ฟิลด์ <code>resume</code>) แทนไฟล์เดิม และตั้ง <code>resume_url</code> เป็น <code>/candidates/<id>/resume</code>
<code>GET /:id/resume</code>	admin หรือผู้สัมภาษณ์ที่เกี่ยวข้อง	ส่งไฟล์เรซูเม่กลับ (ผู้สัมภาษณ์ที่ไม่ได้เกี่ยวข้องได้ 403)

เส้นทาง	สิทธิ์	การทำงาน
DELETE <code>/:id/resume</code>	admin	ลบไฟล์และล้างค่า <code>resume_url</code>
routes/interviews.js → /api/interviews		
GET <code>/</code>	ล็อกอินแล้ว	แอดมินเห็นทั้งหมด ผู้สัมภาษณ์เห็นเฉพาะที่ถูกมอบหมาย (รองรับ <code>status, limit, offset</code>)
POST <code>/</code>	admin	นัดสัมภาษณ์: transaction เพิ่มการสัมภาษณ์ + ผู้สัมภาษณ์ + ปรับสถานะผู้สมัคร บันทึกกิจกรรม และส่งอีเมลยืนยันถึงผู้สมัคร
POST <code>/:id/cancel</code>	admin	เปลี่ยนเป็น Cancelled (เฉพาะที่ยังเป็น Scheduled มิฉะนั้น 409)
POST <code>/:id/notify</code>	admin	ส่งอีเมลเตือนผู้สัมภาษณ์ทุกคนด้วย <code>Promise.allSettled</code> แล้วตอบผลรายคน (<code>sent / logged / failed</code>) และบันทึกกิจกรรม
POST <code>/:id/complete</code>	admin หรือผู้สัมภาษณ์ที่ถูกมอบหมาย	รับคะแนนรายเกณฑ์ (จำนวนเต็ม 1–5) หาชื่อหัวข้อ/เกณฑ์จากฐานข้อมูล รวมคะแนน คำนวณผลด้วย <code>computeEvaluationResult</code> แล้วบันทึกใน transaction และเปลี่ยนสถานะเป็น Completed
PATCH <code>/:id</code>	admin	แก้วันเวลา/ประเภท/สถานที่/บันทึก (มีในฝั่งเซิร์ฟเวอร์ แต่หน้าจอยังไม่มีปุ่มเรียกใช้ — ดูบทที่ 6)
routes/users.js → /api/users		
GET <code>/</code>	ล็อกอินแล้ว	แอดมินเห็นผู้ใช้ทั้งหมด (รองรับ <code>search/limit/offset</code>) ผู้สัมภาษณ์เห็นเฉพาะตนเองและเพื่อนร่วมสัมภาษณ์
POST <code>/</code>	admin	สร้างบัญชีผู้สัมภาษณ์ (role ถูกกำหนดเป็น interviewer เสมอ) เข้ารหัสผ่านผ่าน <code>bcrypt cost 10</code>
PATCH <code>/me/password</code>	ล็อกอินแล้ว	ตรวจรหัสผ่านเดิม รหัสใหม่ต้องยาวอย่างน้อย 8 ตัวอักษร
DELETE <code>/:id</code>	admin	ลบผู้ใช้ — ปฏิเสธถ้าเป็นแอดมินคนสุดท้าย หรือยังถูกมอบหมายในการสัมภาษณ์
DELETE <code>/:id/sessions</code>	admin	บังคับให้ผู้ใช้คนนั้นออกจากระบบทุกอุปกรณ์
routes/evaluation-template.js → /api/evaluation-template		
GET <code>/</code>	ล็อกอินแล้ว	ตอบหัวข้อและเกณฑ์ทั้งหมดตามลำดับ <code>sort_order</code>
POST/PATCH/DELETE ของ <code>/sections</code> และ <code>/criteria</code>	admin	เพิ่ม เปลี่ยนชื่อ ลบ หัวข้อ/เกณฑ์ ทุกการเรียกตอบแม่แบบ ทั้งชุด ใหม่ เพื่อให้หน้าเว็บแทนที่ข้อมูลได้ทันที

เส้นทาง	สิทธิ์	การทำงาน
routes/activity-log.js → /api/activity-log		
GET /	admin	ตอบบันทึกกิจกรรมล่าสุด 200 รายการ (ใหม่สุดก่อน)

3.5 lib/ — ฟังก์ชันช่วยฝั่งเซิร์ฟเวอร์

ไฟล์	ทำอะไร
serialize.js	แปลงแถวจากฐานข้อมูล (ชื่อคอลัมน์แบบ snake_case, ค่าว่างเป็น null) ให้เป็นออบเจกต์แบบที่หน้าเว็บคาดหวัง (camelCase, ค่าว่างเป็น undefined) เช่น <code>serializeInterview</code> ประกอบออบเจกต์ <code>evaluation</code> ซ้อนจากคอลัมน์ <code>eval_*</code> และแถวใน <code>evaluation_scores</code> โดยคำนวณ <code>maxScore</code> = จำนวนเกณฑ์ × 5 ข้อมูลอ่อนไหวเช่น <code>password_hash</code> และ <code>resume_filename</code> จึงไม่หลุดออกไปยังหน้าเว็บ
activityLog.js	<code>logActivity({ actor, action, entityType, entityId, entityLabel, details })</code> เพิ่มแถวลง <code>activity_log</code> พร้อมเวลา ISO ถ้าไม่มีผู้กระทำจะใช้ชื่อ "System" ถูกเรียกจากทุก route ที่เปลี่ยนแปลงข้อมูล
evaluation.js	ไฟล์บรรทัดเดียวที่ re-export <code>computeEvaluationResult</code> จาก <code>src/lib/evaluationThresholds.js</code> เพื่อให้เซิร์ฟเวอร์และหน้าเว็บใช้เกณฑ์ 72%/60% จากแหล่งเดียวกัน
ids.js	<code>generateId(prefix)</code> สร้างรหัสรูปแบบ <code>c-<uuid></code> , <code>i-<uuid></code> , <code>session-<uuid></code> ฯลฯ ด้วย <code>crypto.randomUUID()</code>
mailer.js	<code>sendMail({ to, subject, text })</code> สร้าง transporter ของ nodemailer แบบ lazy จาก SMTP_* ถ้าตั้งค่าไม่ครบจะไม่ส่งจริง แต่เพิ่มพื้เผลลล็อกของเซิร์ฟเวอร์และคืน <code>{ sent: false }</code> (หน้าเว็บแสดงเป็น "logged")
uploads.js	ตั้งค่า multer: เก็บไฟล์ที่ <code>server/data/uploads/resumes</code> (ตอนทดสอบใช้โฟลเดอร์ชั่วคราวของระบบ) ตั้งชื่อไฟล์ใหม่เป็น <code>resume-<uuid>.นามสกุล</code> จำกัด 5 MB และรับเฉพาะ PDF/DOC/DOCX ตาม MIME type

3.6 ชุดทดสอบ (*.test.js และ test/helpers.js)

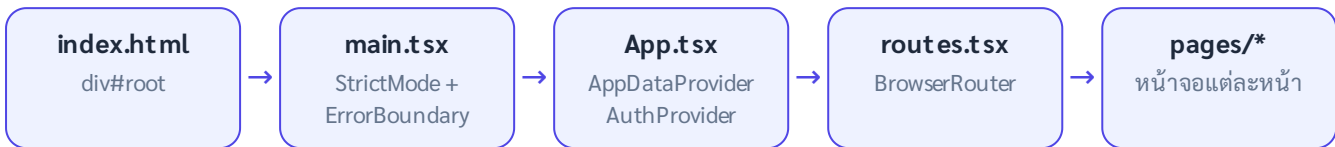
เทสตรันด้วย Vitest กับ supertest เรียก `app` จริงโดยไม่เปิดพอร์ต และใช้ฐานข้อมูล `:memory:` แยกต่อไฟล์ทดสอบ

ไฟล์	ครอบคลุม
test/helpers.js	<code>createUser()</code> สร้างผู้ทดสอบ (bcrypt cost ต่ำเพื่อความเร็ว), <code>clearData()</code> และ <code>clearCandidatesAndInterviews()</code> ล้างข้อมูลระหว่างเทสต์

ไฟล์	ครอบคลุม
routes/auth.test.js	ล็อกอินสำเร็จ/ล้มเหลว และการเปลี่ยนรหัสผ่าน
routes/auth.ratelimit.test.js	อนุญาต 10 ครั้ง ครั้งที่ 11 ถูกบล็อก
routes/auth.sessions.test.js	การเพิกถอน session ด้วยตนเองและโดยแอดมิน และการห้ามผู้ใช้แอดมิน
routes/candidates.test.js	อีเมลซ้ำ การลบแล้วลบการสัมภาษณ์ตาม และสิทธิ์
routes/candidates.resume.test.js	อัปโหลด ดาวน์โหลด ลบเรซูเม่ ชนิดไฟล์ที่ไม่อนุญาต และสิทธิ์ของผู้สัมภาษณ์
routes/interviews.test.js	การเปลี่ยนสถานะผู้สมัครตอนนัด (ไม่ย้อน Offer/Rejected) ข้อมูลไม่ครบ และการยกเลิก/เตือนซ้ำ
lib/evaluation.test.js	เกณฑ์ Hire $\geq 72\%$, Compare 60–71%, Reject $< 60\%$ และกรณีคะแนนเต็มเป็นศูนย์

4 ฝั่งหน้าเว็บ (src/)

4.1 ลำดับการเริ่มทำงานของหน้าเว็บ



ไฟล์	ทำอะไร
src/main.tsx	จุดเข้าของ React: สร้าง root ใน #root ครอบด้วย StrictMode และ ErrorBoundary ระดับบนสุด (ถ้าเกิดข้อผิดพลาดที่ไม่คาดคิดจะแสดงข้อความแทนหน้าจอ) และโหลด index.css
src/App.tsx	ซ้อน provider สองตัว: AppDataProvider (ข้อมูลกลาง) อยู่นอก AuthProvider (สถานะล็อกอิน) เพราะ AuthProvider ต้องเรียก refresh() และ clear() ของข้อมูลกลาง
src/routes.tsx	ประกาศ URL ทั้งหมดด้วย BrowserRouter/Routes/Route และหน้าที่ต้องล็อกอินด้วย RequireAuth รายละเอียดในหัวข้อ 4.2
src/index.css	บรรทัด @import "tailwindcss" (เปิดใช้ Tailwind v4 แบบ CSS-first) และตั้งความสูง 100% ให้ html/body/#root

4.2 การกำหนดเส้นทางและการป้องกันหน้า

URL	ผู้เข้า ได้	หน้า
/login	ทุกคน	LoginPage (ถ้าล็อกอินแล้วจะพาไปหน้าแรกของบทบาท)
/	ล็อกอิน แล้ว	HomeRedirect → /dashboard (admin) หรือ /my-interviews (interviewer) ตาม lib/roleHome.ts
/dashboard, /candidates, /candidates/:candidateId, /scheduling, /interviewers, /evaluation-criteria, /activity-log	admin	ห่อด้วย RequireAuth roles={['admin']}

URL	ผู้เข้า ได้	หน้า
/my-interviews	ล็อกอิน แล้ว	MyInterviewsPage
/candidates/:candidateId/interviews/:interviewId/appraisal	ล็อกอิน แล้ว	AppraisalReportPage (อยู่นอก AppLayout เพื่อให้พิมพ์ได้สะดวก)
*	ล็อกอิน แล้ว	NotFoundPage

`RequireAuth` ทำงานดังนี้: ระหว่างกำลังกู้ session จะไม่แสดงอะไร ถ้ายังไม่ล็อกอินจะ redirect ไป `/login` พร้อมจำหน่ายปลายทางไว้ใน `state.from` (ล็อกอินเสร็จจะกลับมาหน้าเดิม) และถ้าบทบาทไม่ตรงกับ `roles` จะพากลับ `/` ซึ่งเดี๋ยวยังหน้าแรกของบทบาทนั้น

4.3 context/ — ข้อมูลกลางและสถานะล็อกอิน

แอปนี้ไม่ส่งข้อมูลผ่าน props ระหว่างหน้า แต่ให้ทุกคอมโพเนนต์เรียก `useAppData()` และ `useAuth()` ไฟล์ `context` แยกเป็นคู่ (ตัวสร้าง context / ตัว provider) เพื่อให้ผ่านกฎ lint เรื่อง fast-refresh

ไฟล์	ทำอะไร
<code>dataContext.ts</code>	ประกาศ <code>AppDataContextValue</code> (รายการข้อมูล ฟังก์ชัน CRUD ทุกตัว และค่าที่คำนวณ) ชนิดข้อมูลเสริมเช่น <code>EvaluationSubmission</code> และ <code>NotifyRecipientResult</code> และสร้าง <code>AppDataContext</code>
<code>AppDataContext.tsx</code>	หัวใจของหน้าเว็บ — เก็บ <code>candidates</code> , <code>interviews</code> , <code>users</code> , <code>evaluationTemplate</code> ไว้ใน <code>useReducer</code> เดียว (การกระทำ: <code>SET_ALL</code> , <code>RESET</code> , <code>ADD/UPDATE/REMOVE_CANDIDATE</code> , <code>ADD/UPDATE_INTERVIEW</code> , <code>ADD/REMOVE_USER</code> , <code>SET_EVALUATION_TEMPLATE</code>) <code>refresh()</code> ดึง 4 endpoint พร้อมกันด้วย <code>Promise.all</code> ทุกฟังก์ชันแก้ไขข้อมูลเป็น <code>async</code> : เรียก API ก่อน แล้วค่อย <code>dispatch</code> ด้วยข้อมูลที่เซิร์ฟเวอร์ตอบกลับ นอกจากนี้คำนวณค่าอนุพันธ์ใน <code>useMemo</code> : <code>upcomingInterviews</code> , <code>interviewsThisWeek</code> , <code>candidateStatusCounts</code> , <code>interviewTrendByPosition</code> (6 เดือนล่าสุด นับเฉพาะการสัมภาษณ์ที่ Completed แยกตามตำแหน่ง)
<code>useAppData.ts</code>	hook ที่อ่าน context และโยนข้อผิดพลาดถ้าเรียกนอก provider
<code>authContext.ts</code>	ประกาศ <code>AuthContextValue</code> (<code>isAuthenticated</code> , <code>isLoading</code> , <code>currentUser</code> , <code>login</code> , <code>logout</code> , <code>logoutEverywhere</code> , <code>changePassword</code>)

ไฟล์	ทำอะไร
AuthProvider.tsx	ตอนเปิดหน้าเว็บ ถ้ามีโทเค็นใน localStorage จะเรียก GET /auth/me เพื่อที่ผู้ใช้แล้ว refresh() ข้อมูล (ถ้าโทเค็นใช้ไม่ได้จะล้างทิ้ง) login เรียก POST /auth/login เก็บโทเค็น แล้วโหลดข้อมูล logout และ logoutEverywhere ล้างโทเค็น ผู้ใช้ และข้อมูลกลาง
useAuth.ts	hook อ่าน AuthContext

หมายเหตุ

ไฟล์ dataContext.ts กับ AppDataContext.tsx ต่างกันที่ตัวพิมพ์เล็ก-ใหญ่เท่านั้น ใช้ได้บน Windows/macOS ที่ไม่แยกตัวพิมพ์ แต่จะมีปัญหาบนระบบไฟล์ที่แยก (เช่น Linux) ควรเปลี่ยนชื่อหนึ่งในสองไฟล์เมื่อมีโอกาส

4.4 lib/ และ types/

ไฟล์	ทำอะไร
lib/api.ts	ตัวห่อ fetch ทุกคำขอไปหน้าเว็บผ่านที่นี้: ต่อ /api ใส่ header Authorization จากโทเค็นใน localStorage (คือ ims_token) แปลง body เป็น JSON และเมื่อตอบไม่สำเร็จจะโยน Error พร้อมข้อความ error จากเซิร์ฟเวอร์ (จึงเห็นข้อความเช่น "A candidate with that email already exists." บนหน้าจอ) มี api.get/post/patch/delete, api.upload (ส่ง FormData) และ api.downloadBlob (ดาวน์โหลดไฟล์ที่ต้องแนบโทเค็น)
lib/date.ts	ฟังก์ชันวันเวลา: formatDate/Time/DateTime/ShortDateTime, isThisWeek (สัปดาห์เริ่มวันอาทิตย์), sortByDateAsc, monthKey, formatMonthLabel
lib/status.ts	แหล่งกลางของสถานะ: แอปสถานะ → สีของป้าย, รายการตัวเลือกสถานะ/ประเภทการสัมภาษณ์, ข้อความแนะนำเกณฑ์ และ wrapper ของ computeEvaluationResult
lib/evaluationThresholds.js	กำหนด HIRE_THRESHOLD = 0.72, COMPARE_THRESHOLD = 0.6 และ ฟังก์ชัน computeEvaluationResult(total, max) เขียนเป็น .js เพื่อให้ทั้งเซิร์ฟเวอร์และหน้าเว็บ import ใช้ร่วมกัน
lib/roleHome.ts	getHomePath(role) → /my-interviews สำหรับ interviewer นอกนั้น /dashboard

ไฟล์	ทำอะไร
types/*.ts	ชนิดข้อมูล Candidate/CandidateStatus , Interview/Evaluation/... , AppUser/UserRole , EvaluationSection , ActivityLogEntry และ index.ts ที่รวม export ไว้จุดเดียว (ต้องใช้ import type ตามกฎ verbatimModuleSyntax)

4.5 pages/ — หน้าจอ

แต่ละไฟล์คือหนึ่งหน้า เก็บสถานะเฉพาะหน้า (คำค้นหา ตัวกรอง ว่ากำลังเปิด modal ไหน) ส่วนข้อมูลจริงมาจาก useAppData()

ไฟล์	ทำอะไร
LoginPage.tsx	ฟอร์มอีเมล/รหัสผ่าน เรียก login() แล้วพาไปหน้าเดิมที่ตั้งใจเข้า (state.from) หรือหน้าแรกของบทบาท ถ้าผิดพลาดแสดง “Invalid email or password”
DashboardPage.tsx	นำค่าอนุพันธ์จาก context มาแสดงด้วย StatCard x4, StatusFunnel , InterviewList (5 รายการ) และ InterviewTrendChart
CandidatesPage.tsx	ตารางผู้สมัคร กรองฝั่งเบราว์เซอร์ด้วยคำค้น (ชื่อ/อีเมล) และสถานะ ปุ่ม Add Candidate เปิด CandidateFormModal
CandidateDetailPage.tsx	อ่าน :candidateId จาก URL แสดงข้อมูล ปุ่ม Edit/Schedule/Delete (ลบมีหน้าต่างยืนยัน) ResumeUpload CandidateStatusSelect และ ประวัติสัมภาษณ์พร้อมค่าเฉลี่ยคะแนน
SchedulingPage.tsx	รายการสัมภาษณ์ทั้งหมดเรียงตามวัน กรองตามสถานะ และเปิดฟอร์มนัดสัมภาษณ์
InterviewersPage.tsx	ตารางผู้สัมภาษณ์ ปุ่ม Revoke sessions (มีหน้าต่างยืนยัน) และ Remove (ทำทันที) และเปิด InterviewerFormModal
EvaluationCriteriaPage.tsx	แก้แม่แบบเกณฑ์ประเมิน: การ์ดต่อหัวข้อ (SectionCard) และแถวต่อเกณฑ์ (CriterionRow) เปลี่ยนชื่อเมื่อ blur และลบด้วยการยืนยัน
ActivityLogPage.tsx	เรียก GET /activity-log โดยตรงด้วย useEffect (ไม่ผ่าน context) แล้วแสดงตารางแปลง interview.scheduled เป็นข้อความอ่านง่ายด้วย describeAction
MyInterviewsPage.tsx	หน้าแรกของผู้สัมภาษณ์: getInterviewsForInterviewer(currentUser.id) แสดงด้วย InterviewList

ไฟล์	ทำอะไร
AppraisalReportPage.tsx	รายงาน “Interview Evaluation Form” จัดกลุ่มคะแนนตามหัวข้อ แสดงผลรวมลายเซ็น ความเห็น และปุ่ม Print (<code>window.print()</code>) อนุญาตเฉพาะแอดมินหรือผู้สัมภาษณ์ของการสัมภาษณ์นั้น
NotFoundPage.tsx	หน้า 404 สำหรับ URL ที่ไม่ตรงกับ route ใด

4.6 components/ — ชิ้นส่วนที่ใช้ซ้ำ

components/layout — โครงหน้า

ไฟล์	ทำอะไร
AppLayout.tsx	โครงหน้าหลัก: Sidebar + Topbar + DataErrorBanner + พื้นที่เนื้อหา (<code><Outlet/></code>) ห่อด้วย <code>ErrorBoundary</code> ที่ผูกกับ <code>pathname</code> เพื่อให้เปลี่ยนหน้าแล้วล้างข้อผิดพลาดเอง
Sidebar.tsx	เลือกรายการเมนูตามบทบาท (admin 6 เมนู / interviewer 1 เมนู) และไฮไลต์เมนูปัจจุบันด้วย <code>NavLink</code> ซ่อนบนจอเล็ก (แสดงตั้งแต่ breakpoint md)
Topbar.tsx	แสดงชื่อหน้าตาม URL ชื่อผู้ใช้ และปุ่ม Change password / Log out everywhere / Log out
ChangePasswordForm.tsx	modal เปลี่ยนรหัสผ่าน ตรวจสอบความยาว ≥ 8 และการยืนยันตรงกันก่อนเปิดปุ่มบันทึก
DataErrorBanner.tsx	แถบแดงที่ปรากฏเมื่อ context มี error พร้อมปุ่ม Retry เรียก <code>refresh()</code>

components/auth, candidates, interviewers

ไฟล์	ทำอะไร
auth/RequireAuth.tsx , auth/HomeRedirect.tsx	ตัวป้องกัน route ตามบทบาท และตัว redirect จาก / ไปหน้าแรกของบทบาท
candidates/CandidateFormModal.tsx	ฟอร์มเพิ่ม/แก้ไขผู้สมัคร (โหมดตามว่ามี prop candidate หรือไม่)
candidates/CandidateTable.tsx	ตารางผู้สมัคร คลิกแถวเพื่อไปหน้ารายละเอียด
candidates/CandidateStatusSelect.tsx	รายการเลือกสถานะ เรียก <code>updateCandidateStatus</code> ทันทีที่เปลี่ยนค่า
candidates/ResumeUpload.tsx	ปุ่มอัปโหลด/แทนที่/ลบเรซูเม่ ตรวจสอบชนิดไฟล์และขนาด 5 MB ที่ฝั่งเบราว์เซอร์ก่อนส่งขึ้นเซิร์ฟเวอร์ (เซิร์ฟเวอร์ตรวจสอบซ้ำอีกชั้น)

ไฟล์	ทำอะไร
interviewers/InterviewerFormModal.tsx	ฟอร์มสร้างบัญชีผู้สัมภาษณ์ แล้วแสดงอีเมล/รหัสผ่านที่ตั้งให้ส่งต่อ

components/interviews

ไฟล์	ทำอะไร
InterviewList.tsx	ตารางการสัมภาษณ์ที่ใช้ซ้ำใน Dashboard, Scheduling, Candidate Detail และ My Interviews ตัดสินว่าจะแสดงปุ่มใดจากบทบาท (Complete: admin หรือผู้ถูกมอบหมาย, Notify/Cancel: admin) และมีลิงก์ View resume / View report
ScheduleInterviewForm.tsx	modal นัดสัมภาษณ์ (เลือกผู้สมัคร ผู้สัมภาษณ์หลายคน วัน เวลา ระยะเวลา ประเภท สถานที่ บันทึก) ไม่ยอมส่งถ้ายังไม่เลือกผู้สมัครหรือผู้สัมภาษณ์
CompleteInterviewForm.tsx	modal แบบประเมิน: สร้างช่องคะแนนจากแม่แบบเกณฑ์ปัจจุบัน คำนวณรวม/ร้อยละ/ผลแบบสดด้วย computeEvaluationResult ใช้ ScoreGrid และ CircularGauge ปุ่มบันทึกเปิดเมื่อให้คะแนนครบและมีลายเซ็น
NotifyInterviewForm.tsx	modal ส่งอีเมลเตือนผู้สัมภาษณ์ แสดงผลรายคน (sent/logged/failed)

components/dashboard และ components/ui

ไฟล์	ทำอะไร
dashboard/StatCard.tsx	การ์ดตัวเลขสรุป (ป้าย ค่า ไอคอน โทนีสี คำใบ้)
dashboard/StatusFunnel.tsx	แท่งแนวนอนแสดงจำนวนผู้สมัครในแต่ละสถานะ (สเกลตามค่าสูงสุด)
dashboard/InterviewTrendChart.tsx	กราฟเส้นด้วย recharts หนึ่งเส้นต่อหนึ่งตำแหน่งงาน ถ้ายังไม่มีสัมภาษณ์ที่เสร็จสิ้นจะแสดงข้อความว่าง
ui/Button, Card, Input, Table, Badge, EmptyState	ชิ้นส่วนพื้นฐานที่ไม่รู้จักข้อมูลของระบบ (presentation only): ปุ่ม 4 แบบ การ์ด ช่องกรอก/เลือก/ข้อความยาวที่มีป้ายกำกับ ตาราง ป้ายสี และหน้าจอว่าง
ui/badgeStyles.ts , ui/StatusPill.tsx	ตารางสี Tailwind ของป้าย และป้ายสถานะผู้สมัคร/การสัมภาษณ์/ผลประเมินที่ดึงสีจาก lib/status.ts
ui/Modal.tsx	หน้าต่างป๊อปอัพผ่าน portal รองรับปุ่ม Esc, ล็อกโฟกัสด้วย Tab และคืนโฟกัสเมื่อปิด ฟอร์มใน modal จะถูก mount ใหม่ทุกครั้งที่เปิด ({open && <Form/> }) แทนการใช้ useEffect
ui/ScoreGrid.tsx , ui/CircularGauge.tsx	ตารางกดเลือกคะแนน 5-1 แบบ radio และวงกลมแสดงร้อยละด้วย SVG

ไฟล์	ทำอะไร
<code>ui/ResumeLink.tsx</code>	ลิงก์ “View resume” — ถ้าเป็นลิงก์ภายนอกเปิดตามปกติ ถ้าเป็นไฟล์ที่อัปโหลด (ขั้นต้น <code>/candidates/</code>) จะดาวน์โหลดเป็น blob ด้วยโทเค็นแล้วเปิดในแท็บใหม่
<code>ErrorBoundary.tsx</code>	class component ที่จับข้อผิดพลาดตอน render แล้วแสดงข้อความพร้อมปุ่ม “Try again”

5 กระบวนการสำคัญที่ไฟล์ทำงานร่วมกัน

5.1 การเข้าสู่ระบบและการกู้ session

- 1 `LoginPage.tsx` เรียก `login()` ใน `AuthProvider.tsx` ซึ่งส่ง `POST /api/auth/login`
- 2 `routes/auth.js` ตรวจสอบ rate limit → ค้นหาผู้ใช้ตามอีเมล → `bcrypt.compareSync` → สร้างโทเค็นเก็บในตาราง `sessions` (อายุตั้งไว้ยาวมาก) → ตอบ `{ token, user }`
- 3 `api.ts` เก็บโทเค็นใน `localStorage (ims_token)` `AuthProvider` ตั้ง `currentUser` แล้ว `refresh()` ดึงข้อมูลทั้ง 4 ชุด (เซิร์ฟเวอร์กรองข้อมูลให้ผู้สัมภาษณ์เห็นเฉพาะที่เกี่ยวข้อง)
- 4 เปิดหน้าเว็บใหม่: `AuthProvider` พบโทเค็นจึงเรียก `GET /auth/me` ถ้าสำเร็จได้ผู้ใช้กลับมาโดยไม่ต้องล็อกอินซ้ำ ถ้าไม่สำเร็จ (เช่นถูก revoke) โทเค็นถูกล้างและกลับสู่หน้า Sign in

5.2 การบันทึกผลสัมภาษณ์

- 1 `CompleteInterviewForm.tsx` อ่านแม่แบบเกณฑ์จาก context สร้างช่องคะแนนต่อเกณฑ์ และคำนวณผลสดด้วย `lib/status.ts` → `evaluationThresholds.js`
- 2 เมื่อกดบันทึก ส่ง `{ scores: [{ criterionId, score }], notes, interviewerSignature, interviewerPosition }` ไป `POST /api/interviews/:id/complete`
- 3 `routes/interviews.js` ตรวจสอบสิทธิ์ (admin หรือผู้ถูกมอบหมาย) และสถานะ (ต้องเป็น Scheduled) หาชื่อหัวข้อ/เกณฑ์จาก `criterionId` ตรวจสอบว่าคะแนนเป็นจำนวนเต็ม 1–5 คำนวณผลซ้ำที่เซิร์ฟเวอร์ไม่เชื่อคำจากหน้าเว็บ
- 4 ใน transaction: ลบคะแนนเดิม (ถ้ามี) เพิ่มแถวใน `evaluation_scores` อัปเดตคอลัมน์ `eval_*` และตั้งสถานะ Completed แล้วบันทึกกิจกรรม “Result: Hire (75/100)”
- 5 เซิร์ฟเวอร์ตอบการสัมภาษณ์ที่ผ่าน `serializeInterview` (มี `evaluation` ซ้อนอยู่) → context dispatch `UPDATE_INTERVIEW` → ตาราง ปุ่ม View report และหน้า `AppraisalReportPage` อัปเดตทันที

5.3 เรซูเม่: อัปโหลดและเปิดดู

- 1 `ResumeUpload.tsx` ตรวจสอบไฟล์เบื้องต้น แล้ว `uploadResume()` ส่ง `FormData` (ฟิลด์ `resume`) ผ่าน `api.upload`

- 2 `lib/uploads.js` (multer) กรองชนิดไฟล์ ตรวจสอบขนาด และบันทึกเป็นไฟล์ที่ตั้งชื่อใหม่ใน `server/data/uploads/resumes`
- 3 `routes/candidates.js` ลบไฟล์เก่า เก็บชื่อไฟล์จริงใน `resume_filename` (ภายในเท่านั้น) และตั้ง `resume_url` เป็น `/candidates/<id>/resume` ซึ่งเป็นค่าที่หน้าเว็บเห็น
- 4 เมื่อกด View resume `ResumeLink.tsx` เห็นว่า URL ขึ้นต้นด้วย `/candidates/` จึงเรียก `api.downloadBlob` (แนบโทเค็น) เพราะลิงก์ธรรมดาแนบ header ไม่ได้ แล้วเปิด blob ในแท็บใหม่ — เซิร์ฟเวอร์ตรวจว่าเป็นแอดมินหรือผู้สัมภาษณ์ที่เกี่ยวข้องกับผู้สมัครคนนั้นก่อนส่งไฟล์

5.4 อีเมลแจ้งเตือน

อีเมลมีสองแบบ ทั้งสองผ่าน `lib/mailer.js` และเนื้อหาสร้างใน `routes/interviews.js` โดยแสดงเวลาตามเขตเวลา `APP_TIMEZONE`

- **ยืนยันถึงผู้สมัคร** — ส่งอัตโนมัติหลังนัดสัมภาษณ์สำเร็จ แบบไม่รอผล (ถ้าล้มเหลวไม่ทำให้การนัดล้มเหลว) และบันทึกผลใน Activity Log (`candidate.confirmation_sent` / `confirmation_failed`)
- **เตือนผู้สัมภาษณ์** — ส่งเมื่อแอดมินกด Notify ส่งพร้อมกันทุกคนและรายงานผลรายคน (`sent` / `logged` / `failed`)
- ถ้าไม่ได้ตั้งค่า SMTP อีเมลจะถูกพิมพ์ลงล็อกของเซิร์ฟเวอร์เท่านั้น

6 ข้อสังเกตและข้อควรระวังจากการอ่านโค้ด

รายการด้านล่างได้จากการอ่านโค้ดโดยตรง ข้อแรกผมยืนยันด้วยการจำลองแล้ว ส่วนข้ออื่นเป็นข้อสังเกตเชิงออกแบบ

1. Docker image ที่สร้างจาก Dockerfile ปัจจุบันน่าจะเริ่มทำงานไม่ได้

`server/lib/evaluation.js` import ไฟล์จาก `../../src/lib/evaluationThresholds.js` แต่ขั้น runtime ของ Dockerfile คัดลอกเพียง `server/` และ `dist/` ไม่มีโฟลเดอร์ `src/` เมื่อผมจำลองโครงสร้างนี้ (ไม่มี `src`) การ import ล้มเหลวด้วย `ERR_MODULE_NOT_FOUND` ซึ่งจะทำให้เซิร์ฟเวอร์ใน container ล้มตั้งแต่เริ่ม เพราะ `routes/interviews.js` โหลดไฟล์นี้ CI ตรวจเฉพาะขั้น `docker build` ไม่ได้รัน container จึงไม่พบปัญหา

แนวทางแก้: เพิ่ม `COPY src/lib/evaluationThresholds.js ./src/lib/evaluationThresholds.js` ในขั้น runtime หรือย้ายไฟล์ค่าเกณฑ์ไปไว้ใน `server/lib` แล้วให้หน้าเว็บ import จากที่เดียวกัน และควรเพิ่มขั้นตอนใน CI ที่รัน container แล้วเรียก `/healthz`

#	ข้อสังเกต	รายละเอียด
2	เกณฑ์ 72%/60% ถูกเขียนซ้ำใน <code>server/seed.js</code>	ฟังก์ชัน <code>computeResult</code> ใน <code>seed</code> ใช้ตัวเลขของตัวเอง ไม่ได้ import จาก <code>evaluationThresholds.js</code> ถ้าปรับเกณฑ์ในภายหลัง ข้อมูลตัวอย่างจะไม่ตามไปด้วย (ไม่กระทบระบบจริง)
3	session แทบไม่หมดอายุ	อายุโทเค็นตั้งไว้ราว 100 ปี และเก็บใน <code>localStorage</code> จึงต้องพึ่ง <code>Log out / Revoke sessions</code> เพื่อเพิกถอน (ตามที่ <code>commit</code> ล่าสุดเพิ่มไว้) หากต้องการความรัดกุมขึ้นอาจกำหนดอายุสั้นลง
4	ฝั่งเซิร์ฟเวอร์และ context มีการแก้ไขการสัมภาษณ์ที่หน้าจอไม่ได้ใช้	<code>PATCH /api/interviews/:id</code> และ <code>updateInterview()</code> ใน <code>context</code> พร้อมใช้ แต่ไม่มีคอมโพเนนต์เรียกใช้ จึงยังเลื่อนเวลานัดผ่านหน้าจอไม่ได้ เพิ่มฟอร์มแก้ไขได้โดยไม่ต้องแก้ฝั่งเซิร์ฟเวอร์
5	ไม่มีการเปลี่ยนรหัสผ่านให้ผู้อื่น	<code>routes/users.js</code> มีเพียงการเปลี่ยนรหัสผ่านของตนเอง (<code>/me/password</code>) แอดมินรีเซ็ตให้ผู้สัมภาษณ์ผ่านระบบไม่ได้
6	ฟอร์มสร้างผู้สัมภาษณ์แสดงรหัสผ่านเป็นข้อความธรรมดา	<code>InterviewerFormModal.tsx</code> ใช้ช่อง <code>type="text"</code> และแสดงรหัสผ่านซ้ำหลังสร้างเสร็จ เพื่อให้แอดมินส่งต่อได้ (เซิร์ฟเวอร์ไม่ส่งรหัสผ่านกลับ) เป็นการแลกความสะดวกกับความเสี่ยงถูกแอบดู
7	ปุ่ม Cancel และ Remove ไม่ถามยืนยัน	<code>InterviewList.tsx</code> (Cancel) และ <code>InterviewersPage.tsx</code> (Remove) เรียกใช้ทันที ต่างจากการลบผู้สมัครและ <code>Revoke sessions</code> ที่มี <code>window.confirm</code>
8	ไฟล์ context ชื่อต่างกันที่ตัวพิมพ์เท่านั้น	<code>dataContext.ts</code> กับ <code>AppDataContext.tsx</code> (ดูหัวข้อ 4.3) อาจเกิดปัญหาบนระบบไฟล์ที่แยกตัวพิมพ์เล็ก-ใหญ่

#	ข้อสังเกต	รายละเอียด
9	เทสยังไม่ครอบคลุมหน้าเว็บ	ชุดเทสต์มีเฉพาะฝั่งเซิร์ฟเวอร์ ฝั่งหน้าเว็บอาศัยการตรวจชนิดข้อมูลตอน <code>npm run build</code> เท่านั้น และ <code>npm run lint</code> ไม่ตรวจโฟลเดอร์ <code>server/</code>

7 ต้องการแก้เรื่องนี้ ควรเปิดไฟล์ไหน

ตารางลัดสำหรับนักพัฒนา เรียงตามงานที่พบบ่อย

ต้องการ	ไฟล์ที่ต้องแก้
เปลี่ยนเกณฑ์ Hire/Compare/Reject	<code>src/lib/evaluationThresholds.js</code> (ใช้ร่วมกันทั้งสองฝั่ง) และแก้ข้อความแนะนำใน <code>src/lib/status.ts</code> (<code>evaluationResultGuide</code>) รวมถึง <code>server/seed.js</code> ถ้าต้องการให้ข้อมูลตัวอย่างตรงกัน
เพิ่มสถานะผู้สมัครใหม่	<code>server/schema.sql</code> (CHECK) — ฐานข้อมูลเดิมต้องย้ายข้อมูลเอง เพราะ <code>CREATE TABLE IF NOT EXISTS</code> ไม่แก้ตารางเดิม, <code>src/types/candidate.ts</code> , <code>src/lib/status.ts</code> (สี + ตัวเลือก), <code>AppDataContext.tsx</code> (<code>STATUS_LIST</code>)
เพิ่มประเภทการสัมภาษณ์ใหม่	<code>server/schema.sql</code> (CHECK), <code>src/types/interview.ts</code> , <code>interviewTypeOptions</code> ใน <code>src/lib/status.ts</code>
เพิ่มช่องข้อมูลของผู้สมัคร	คอลัมน์ใน <code>schema.sql</code> + <code>ensureColumn</code> ใน <code>db.js</code> สำหรับฐานข้อมูลเดิม, <code>routes/candidates.js</code> (<code>insert/patch</code>), <code>serialize.js</code> , <code>types/candidate.ts</code> , <code>CandidateFormModal.tsx</code> และหน้าที่แสดงผล
เพิ่ม endpoint ใหม่	สร้างหรือแก้ไฟล์ใน <code>server/routes/</code> ผูกใน <code>server/index.js</code> ใช้ <code>requireAuth/requireRole</code> เรียก <code>logActivity</code> และเพิ่มไฟล์ <code>*.test.js</code>
เพิ่มหน้าใหม่ในเมนู	ไฟล์ใน <code>src/pages/</code> , ประกาศ route ใน <code>src/routes.tsx</code> , เพิ่มเมนูใน <code>Sidebar.tsx</code> และชื่อหัวหน้าใน <code>titles</code> ของ <code>Topbar.tsx</code>
เปลี่ยนสิทธิ์ของบทบาท	ฝั่งเซิร์ฟเวอร์ (ตัวจริง): <code>requireRole</code> ใน route นั้น ๆ, ฝั่งหน้าเว็บ (ตัวช่วยซ่อนปุ่ม/หน้า): <code>routes.tsx</code> , <code>Sidebar.tsx</code> , <code>InterviewList.tsx</code>
ปรับสีป้ายสถานะ	<code>src/lib/status.ts</code> (แมปสถานะ → สี) และ <code>src/components/ui/badgeStyles.ts</code> (นิยามสี)

ต้องการ	ไฟล์ที่ต้องแก้
ปรับข้อความอีเมล	<code>buildNotificationEmail</code> และ <code>buildCandidateConfirmationEmail</code> ใน <code>server/routes/interviews.js</code>
เปลี่ยนขนาด/ชนิดไฟล์เรซูเม่ที่รับ	<code>server/lib/uploads.js</code> (ตัวจริง) และค่าคงที่ใน <code>src/components/candidates/ResumeUpload.tsx</code> (ตรวจสอบล่วงหน้า)
ตั้งค่าอีเมล เขตเวลา CORS	ไฟล์ <code>.env</code> (ดูแม่แบบใน <code>.env.example</code>)